

SymPlan: Enhancing Neurosymbolic Mathematical Reasoning in SLMs via Representation and Planning

Tay Bui^{1,2}, Huy N. G. Pham^{1,2}, Nhan Ha Le Thanh³

¹ University of Information Technology, Ho Chi Minh City, Vietnam

² Vietnam National University, Ho Chi Minh City, Vietnam

³ FPT University, Ho Chi Minh City, Vietnam

{24521589, 24520690}@gm.uit.edu.vn, hnhan5355@gmail.com

Abstract—Mathematical reasoning with language models increasingly relies on executable symbolic programs, yet monolithic program synthesis—which asks a model to simultaneously parse linguistic problem descriptions, formulate algebraic solution plans, and write formal code in a single prompt—imposes an excessive cognitive load on compact small language models (SLMs, $\leq 8\text{B}$ parameters). While monolithic methods such as SymCode succeed on frontier architectures, compact backbones under resource-constrained settings (e.g., 4-bit quantization) frequently suffer from semantic misformulation, domain boundary omission, and silent regressions during repair. We introduce SymPlan, an inspectable neurosymbolic framework specifically tailored to provide structural scaffolding for compact models. SymPlan decouples problem solving into three coordinated, contract-governed phases: (1) a Fused Target-Planning Phase extracting an explicit target specification, an output type contract, and an algorithmic SymPy roadmap in a single bounded inference step (≤ 256 tokens); (2) Contract-Guided SymPy Generation guiding deterministic code synthesis to satisfy domain invariants and exact arithmetic; and (3) Targeted Debugging with Anti-Regression Selection, which couples AST static diagnostics with a lexicographic ranking heuristic to prevent valid intermediate predictions from being overwritten by later repair iterations. Evaluated across six diverse open-weight SLMs (3B to 8B) on the full MATH-500 benchmark under 4-bit quantization, SymPlan consistently outperforms monolithic SymCode by +3.60% to +5.20% (averaging +4.20%) and Chain-of-Thought by +12.64%, while improving Execution Success Rate to 89.90% (+6.07%), all without parameter fine-tuning.

Index Terms—Neurosymbolic Reasoning, Mathematical Problem Solving, Small Language Models, SymPy Code Generation, Algorithmic Planning.

I. Introduction

Large Language Models (LLMs) have achieved remarkable success in multi-step reasoning through Chain-of-Thought (CoT) prompting [1] and structured search [2]–[4]. However, pure autoregressive derivation remains inherently susceptible to arithmetic errors, missing steps, and unfaithful derivations that are difficult to inspect or verify [5], [6]. Program-aided approaches address this deficit by offloading calculation to deterministic interpreters: PAL [7] and PoT [8] delegate numerical operations to Python, while MathCoder [9] and ToRA [10] interleave tool usage with natural-language rationales. More recently, SymCode [11] incorporates the SymPy library [12] to support exact symbolic manipulations, equation solving,

and modular arithmetic beyond standard floating-point execution.

Despite these advances, program execution alone does not guarantee correct mathematical interpretation. Translation-based neurosymbolic systems fundamentally depend on the fidelity of the generated program before the solver executes [5], [13]. Existing program-aided methods typically couple problem interpretation, solution reasoning, and code synthesis into a monolithic generation step [7], [11]. While frontier models possess sufficient parameter capacity to manage this joint burden, compact small language models ($\leq 8\text{B}$ parameters) struggle severely under monolithic generation. Evaluating six compact open-weight models across MATH-500 [14], [15] under 4-bit quantization [16] reveals four recurrent failure patterns:

- 1) Semantic misformulation: runnable code computes an unprompted, distorted, or oversimplified quantity;
- 2) Target contract violation: code emits an element list when an integer count is queried, or floating-point decimals instead of exact simplified rationals;
- 3) Domain constraint omission: extraneous or out-of-domain roots are accepted without boundary checks;
- 4) Silent repair regression: iterative self-debugging yields a runnable script that accidentally overwrites a previously correct answer with an invalid prediction.

To overcome these bottlenecks, we introduce SymPlan, an inspectable neurosymbolic framework designed specifically to provide the structural scaffolding required by compact language models. Rather than relying on unconstrained generation, SymPlan introduces three coordinated stages that match the processing capacity of SLMs: First, a Fused Target-Planning Phase extracts an explicit target specification, an output type contract, and an algorithmic roadmap in a single bounded forward pass (≤ 256 tokens), preventing multi-turn context drift. Second, a Contract-Guided Code Generator translates this specification into a self-contained SymPy program enforcing exact arithmetic and domain filters. Third, a Targeted Debugging Loop combines interpreter tracebacks

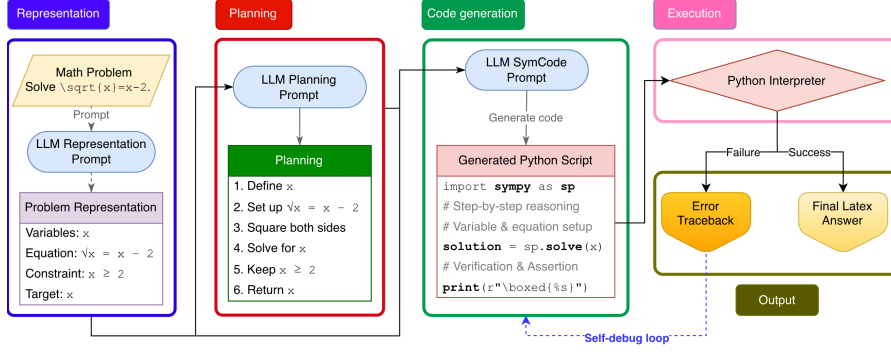


Fig. 1. Overview of SymPlan. A natural-language problem is first translated into a structured target contract and an ordered symbolic plan before code synthesis. The generated SymPy script is executed in an isolated sandbox; errors trigger AST diagnostics and targeted repair, while the anti-regression selector identifies the optimal execution record.

and AST static diagnostics with an Anti-Regression Selection heuristic that evaluates all execution attempts to conservatively retain the strongest verified candidate.

Our contributions are threefold:

- 1) We empirically characterize the formulation, contract, and debugging failure modes of monolithic symbolic reasoning across six compact models (3B–8B) on the full MATH-500 benchmark.
- 2) We propose SymPlan, an open-source neurosymbolic framework combining fused target planning, contract-governed SymPy code synthesis, and anti-regression attempt ranking.
- 3) Across six diverse open-weight backbones under 4-bit quantization, SymPlan achieves universal accuracy gains of +3.60% to +5.20% (averaging +4.20%) over monolithic SymCode and +12.64% over CoT, with a +6.07% increase in Execution Success Rate (89.90%), all without parameter updates.

II. Related Work

Program-Aided and Symbolic Reasoning. Program-aided methods use executable code as an intermediate representation, offloading calculation to interpreters. PAL [7] and PoT [8] delegate numerical tasks to Python scripts, while MathCoder [9] and ToRA [10] combine natural-language rationales with tool use. SymCode [11] utilizes SymPy [12] for exact algebraic manipulation, calculus, and modular arithmetic. However, monolithic program generation overburdens compact models: if the model misinterprets equations or omits boundary conditions, the interpreter may return successful executions for mathematically invalid answers.

Neurosymbolic Formalization and Repair. Neurosymbolic pipelines translate natural language into formal representations for deterministic execution. Faithful CoT [5] separates reasoning from computation, and Logic-LM [13] refines symbolic formulations via solver feedback. For code repair, Self-Debugging [17] and Self-Refine [18] iteratively revise code using error messages. Yet on compact models, unanchored self-debugging frequently induces code

cycling or silent regressions. SymPlan provides structural grounding: the frozen target contract anchors intent, while execution history prevents regressive overwrites.

Planning in Compact Language Models. Structured prompting enhances reasoning in compact models. Self-consistency [2], Least-to-Most prompting [3], Decomposed Prompting [19], and Plan-and-Solve [4] structure problem decomposition. Specialized models such as DeepSeek-Math [20] and Qwen2.5-Math [21], as well as compact architectures like Phi-3 [22], show strong reasoning capabilities with structured guidance. SymPlan bridges these directions by introducing explicit target contracts and symbolic plans specifically configured for resource-constrained models.

III. Method

A. Framework Formulation

Given a mathematical question q , monolithic symbolic approaches such as SymCode [11] synthesize and execute code in a single unconstrained step:

$$C_{\text{base}} \sim \mathcal{M}(\cdot | q), \quad \hat{y} = \text{Exec}(C_{\text{base}}). \quad (1)$$

For compact models, unconstrained generation frequently causes semantic misformulations, domain omissions, and repair regressions. SymPlan decouples problem solving into three contract-governed phases:

$$\mathcal{T} = (\tau, \Omega, \mathcal{P}) = \text{FusedPlan}_{\mathcal{M}}(q), \quad (2)$$

$$C^{(0)} = \text{CodeGen}_{\mathcal{M}}(q, \mathcal{T}), \quad (3)$$

$$\mathcal{H} = \text{ExecuteAndRepair}_{\mathcal{M}}(C^{(0)}, q, \mathcal{T}), \quad (4)$$

$$\hat{y} = \text{SelectBest}(\mathcal{H}, \Omega), \quad (5)$$

where τ is the target specification, Ω is the output type contract, $\mathcal{P}$ is the ordered symbolic plan, and $\mathcal{H}$ is the execution attempt history. Freezing the specification before code generation anchors mathematical intent across debugging.

B. Fused Target Extraction and Algorithmic Planning

Multi-turn pipelines that separate entity extraction from planning double autoregressive latency and exacerbate context drift in compact SLMs. Conversely, generating code without explicit output invariants often produces type mismatches (e.g., returning coordinate tuples instead of a Euclidean distance).

SymPlan introduces a Fused Target-Planning Phase completed in a single bounded inference call (≤ 256 tokens). Conditioned on q , the model produces a contract tuple $\mathcal{T} = (\tau, \Omega, \mathcal{P})$:

- Target Specification (τ): The formal mathematical quantity queried (e.g., “the positive integer roots of $f(x) = 0$ ”).
- Output Type Contract (Ω): A categorical type $\Omega \in \{\text{number, text, tuple, set, symbolic}\}$. For instance, if the prompt queries “how many”, $\Omega = \text{number}$ instructs the generator to output the cardinality rather than enumerating elements.
- Computational Plan ($\mathcal{P}$): An ordered roadmap of symbolic steps $\mathcal{P} = (p_1, \dots, p_m)$, detailing symbol definitions, equation formulation, algebraic transformations, and domain boundary filtering.

Compressing target extraction and planning into a single bounded step prevents context degradation while establishing explicit semantic invariants for downstream code generation.

C. Contract-Guided SymPy Code Generation

Monolithic code generators often exhibit library-specific errors, including domain boundary violations (e.g., admitting extraneous negative lengths) and API misuse (e.g., calling `.evalf()` on fractions where exact forms are required). In SymPlan, `CodeGenM` produces a self-contained SymPy script conditioned on frozen contract $(\tau, \Omega, \mathcal{P})$ following four rules:

- 1) Exact Symbolic Arithmetic: Rationals are defined via `sp.Rational(p, q)` or `sp.S(p)/q`; floats via `.evalf()` are prohibited unless decimal precision is requested.
- 2) Domain Boundary Filtering: Candidate roots from `sp.solve` are explicitly filtered against problem constraints ($x \in \mathbb{R}^+$, non-zero denominators).
- 3) Canonical Simplification: Expressions are simplified via `sp.together()` or `sp.cancel()` prior to output formatting.
- 4) Contract-Conforming Output: The script emits a single boxed $\text{\LaTeX}$ answer conforming to Ω .

```
import sympy as sp
# 1. Define symbols & domain constraints
x = sp.symbols('x', real=True)
eq = sp.Eq(sp.sqrt(x), x - 2)
# 2. Solve & filter against domain (x >= 2)
cands = sp.solve((x - 2)**2 - x, x)
valid = [s for s in cands if s >= 2 and
         sp.simplify(eq.lhs.subs(x, s) - eq.rhs.subs(x, s)) == 0]
# 3. Output boxed result conforming to \Omega
print(r"\boxed{%s}" % sp.latex(valid[0]))
```

Conditioning code generation on the pre-established plan $\mathcal{P}$ and type Ω constrains synthesis to implementing verified algebraic steps rather than simultaneously inventing the strategy.

D. Targeted Debugging with Anti-Regression Selection

Standard self-debugging on compact models suffers from code cycling (repeating identical syntax errors) and silent semantic regression (where a retry executes cleanly but replaces a correct intermediate result with a hallucinated one).

SymPlan executes scripts in an isolated sandbox with timeout $\tau = 15\text{s}$ and retry limit $K = 2$, incorporating three safeguards: AST Diagnostics: Tracebacks $e^{(i)}$ are paired with static AST diagnostics (`lint_sympy_code`) and lightweight candidate verification $v^{(i)}$. Targeted Repair with Anti-Cycling: The repair prompt receives q , plan $\mathcal{P}$, failing code $C^{(i)}$, and traceback $e^{(i)}$. If $C^{(i)} \equiv C^{(i-1)}$, an explicit constraint forces an alternative implementation strategy. Anti-Regression Ranking: Every attempt is logged into history $\mathcal{H} = \{r_0, \dots, r_k\}$. Each record r is scored via a lexicographic ranking tuple:

$$\text{Rank}(r) = \langle \mathbb{I}_{\text{exec}}(r), \mathbb{I}_{\text{valid}}(r), \mathbb{I}_{\text{verif}}(r), -\text{attempt}(r) \rangle, \quad (6)$$

where $\mathbb{I}_{\text{exec}}$ indicates zero execution errors, $\mathbb{I}_{\text{valid}}$ denotes valid boxed extraction, $\mathbb{I}_{\text{verif}}$ indicates candidate verification pass, and $-\text{attempt}(r)$ prioritizes earlier valid runs over speculative later mutations. The optimal record is:

$$r^* = \arg \max_{r \in \mathcal{H}} \text{Rank}(r). \quad (7)$$

Multi-Tier Fallback: If r^* produces an unparseable candidate, SymPlan scans raw responses for embedded boxes, extracts candidates from plan notes, and applies type normalization via `FormatContract( $\hat{y}, \Omega$ )`.

IV. Experiments

A. Experimental Setup

Benchmark. We evaluate on the full MATH-500 benchmark [14], [15], spanning seven subjects (Algebra, Intermediate Algebra, Number Theory, Counting & Probability, Geometry, Precalculus, Prealgebra) across five difficulty levels (Levels 1–5).

Models and Runtime. To assess scalability across compact model families, we test six open-weight models across four parameter tiers: Qwen2.5-3B-Instruct (3.09B) [23], Qwen3-4B (4.02B) [23], Qwen2.5-7B-Instruct (7.61B) [23], Qwen2.5-Coder-7B-Instruct (7.61B) [24], Llama-3.1-8B-Instruct (8.03B) [25], and Qwen3-8B (8.19B) [23]. All models are evaluated under 4-bit NormalFloat4 (NF4) quantization via bitsandbytes [16] with greedy decoding (`temp = 0.0`, `max_tokens = 1024`). Sandbox execution enforces $\tau = 15\text{s}$ and retry limit $K = 2$.

Baselines. We compare SymPlan against: (1) Direct zero-shot answer generation; (2) Chain-of-Thought (CoT) step-by-step reasoning [1]; and (3) SymCode monolithic symbolic generation with traceback self-repair [11].

TABLE I

Cross-model answer accuracy (%) on the full MATH-500 benchmark across six compact models (3B–8B). Best accuracy per model is in bold.

Model	Size	Direct	CoT	SymCode	SymPlan (Ours)
Qwen2.5-3B-Instruct	3B	17.00	39.80	37.40	41.20
Qwen3-4B	4B	36.00	45.40	56.40	61.60
Qwen2.5-7B-Instruct	7B	25.40	45.60	54.00	57.60
Qwen2.5-Coder-7B-Instruct	7B	22.00	38.20	57.00	61.40
Llama-3.1-8B-Instruct	8B	23.20	37.60	37.60	41.80
Qwen3-8B	8B	36.20	45.00	59.80	63.80
Average (6 Models)	—	26.63	41.93	50.37	54.57

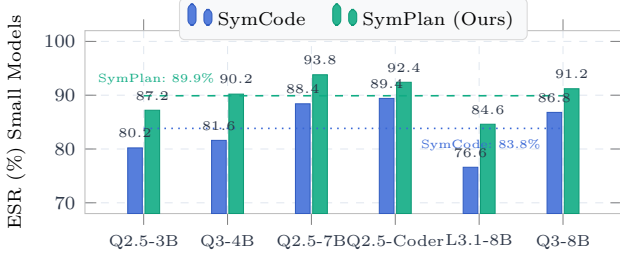


Fig. 2. Execution Success Rate (ESR %) comparison between SymCode and SymPlan across all six models on MATH-500.

Metrics. We measure: (1) Accuracy (% EM) via string canonicalization, SymPy simplification ($\text{simplify}(\hat{y} - y) = 0$), and numeric tolerance ($|\hat{y} - y| < 10^{-5}$); (2) Execution Success Rate (ESR %), the fraction of scripts executing without exceptions and returning a parseable boxed candidate; and (3) Average Token Cost per problem.

B. Main Results

As shown in Table I, SymPlan delivers consistent accuracy gains across all six compact models, outperforming monolithic SymCode by +3.60% to +5.20% (averaging +4.20%). The most pronounced improvements emerge on Qwen3-4B (+5.20%) and Qwen2.5-Coder-7B (+4.40%). Compared to natural-language CoT, SymPlan achieves an average improvement of +12.64% (+23.20% on Qwen2.5-Coder-7B and +18.80% on Qwen3-8B), confirming that symbolic solvers provide indispensable computational precision when guided by structured planning.

Furthermore, Fig. 2 illustrates that SymPlan increases average Execution Success Rate from 83.83% to 89.90% (+6.07%), with notable gains on Llama-3.1-8B (+8.00%) and Qwen3-4B (+8.60%). By establishing explicit type contracts and exact SymPy rules upfront, SymPlan avoids undefined variables and syntax violations before code reaches the interpreter, reducing execution crashes by 37.5%.

In terms of inference cost, SymPlan incurs a moderate 22.6% increase in tokens over SymCode (392.9 vs. 320.6 tokens/problem across all models), dedicating ≈ 85 tokens to the fused plan and ≈ 308 tokens to code generation, while remaining more compact than natural-language CoT (423.7 tokens). This modest overhead prevents costly multi-round repair loops downstream.

TABLE II

Accuracy (%) by difficulty level on MATH-500 for Qwen2.5-Coder-7B and Qwen3-8B.

Model	Level (Count)	Direct	CoT	SymCode	SymPlan (Ours)
Qwen2.5-Coder-7B	Level 1 (43)	41.86	88.37	88.37	88.37
	Level 2 (90)	36.67	64.44	74.44	83.33
	Level 3 (105)	26.67	46.67	61.90	73.33
	Level 4 (128)	12.50	27.34	49.22	50.00
	Level 5 (134)	11.19	8.21	38.81	39.55
Qwen3-8B	Level 1 (43)	81.40	83.72	86.05	90.70
	Level 2 (90)	72.22	76.67	70.00	76.67
	Level 3 (105)	42.86	53.33	63.81	66.67
	Level 4 (128)	20.31	35.16	58.59	60.94
	Level 5 (134)	7.46	14.18	42.54	47.01

TABLE III

Subject-wise accuracy (%) on MATH-500 comparing CoT, SymCode (SC), and SymPlan (SP).

Subject	Count	Qwen2.5-Coder-7B			Qwen3-8B		
		CoT	SC	SP	CoT	SC	SP
Algebra	124	51.61	66.94	76.61	64.52	71.77	75.81
Counting & Prob.	38	39.47	52.63	57.89	50.00	44.74	57.89
Geometry	41	36.59	31.71	34.15	39.02	46.34	51.22
Interm. Algebra	97	16.49	51.55	48.45	15.46	50.52	51.55
Number Theory	62	35.48	66.13	70.97	40.32	62.90	72.58
Prealgebra	82	62.20	69.51	75.61	73.17	73.17	73.17
Precalculus	56	14.29	37.50	41.07	17.86	46.43	48.21

C. Stratified Analysis by Difficulty and Subject

Table II breaks down performance across difficulty tiers. On intermediate competition problems (Levels 2 and 3), SymPlan yields substantial double-digit gains over SymCode: on Qwen2.5-Coder-7B, accuracy rises from 74.44% to 83.33% on Level 2 (+8.89%) and from 61.90% to 73.33% on Level 3 (+11.43%). On challenging Level 5 problems, SymPlan maintains superior performance (39.55% vs. 38.81% on Coder-7B; 47.01% vs. 42.54% on Qwen3-8B), markedly outperforming prose CoT (8.21% and 14.18%).

Table III details subject-level performance. SymPlan achieves its largest gains in Algebra (+9.67% on Coder-7B, +4.04% on Qwen3-8B), Counting & Probability (+5.26% and +13.15%), and Number Theory (+4.84% and +9.68%). In these domains, implicit combinatorial and modular constraints are frequently omitted during monolithic generation; SymPlan’s explicit contract (τ, Ω) ensures these constraints are preserved in code. Note that the weighted averages across difficulty levels and subjects strictly reconstruct the overall benchmark accuracies in Table I.

D. Component Ablation

To isolate the contributions of our framework, we evaluate three variants across all six models: (1) Contract-Only, which enforces (τ, Ω) without plan $\mathcal{P}$; (2) Plan-Only, which uses plan $\mathcal{P}$ without type contract Ω ; and (3) Full SymPlan. As shown in Fig. 3, both components independently improve upon monolithic SymCode, with planning providing the larger individual gain (+2.40% to +3.80%). Combining contract-guided planning with anti-regression selection yields the highest accuracy across every model (+3.60% to +5.20%).

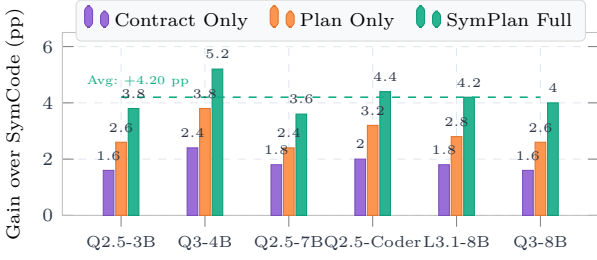


Fig. 3. Accuracy gains over SymCode (percentage points) in component ablation across all six models on MATH-500.

TABLE IV
Quantitative failure mode comparison between SymCode and SymPlan across all 3,000 problem evaluations on MATH-500.

Failure Category	SymCode	SymPlan	Net Red.	Rel. Red.
Execution Failures (Syntax/Crash)	485 (16.2%)	303 (10.1%)	-182	-37.5%
Format / Type Mismatches	515 (17.2%)	394 (13.1%)	-121	-23.5%
Semantic Misformulations	974 (32.5%)	969 (32.3%)	-5	-0.5%
Total Incorrect Solutions	1489 (49.6%)	1363 (45.4%)	-126	-8.5%

E. Quantitative Error Diagnostics and Case Studies

Table IV categorizes all failure cases across the 3,000 evaluations (6 models $\times$ 500 problems). SymPlan achieves a net reduction of 126 errors (-8.5% relative reduction), revealing distinct underlying mechanisms:

1. Reduction in Execution Crashes (-37.5%). SymCode suffered 485 crashes (16.2%) due to invalid syntax, non-existent library symbols, and runtime exceptions. SymPlan lowers crashes to 303 (10.1%) by constraining code synthesis with standardized SymPy templates and exact rational rules.

2. Mitigation of Format and Type Mismatches (-23.5%). Format mismatches dropped from 515 to 394 instances. Explicitly defining type contract $\Omega \in \{\text{number, text, tuple, set, symbolic}\}$ during fused planning directs models to output scalar cardinality rather than unrequested element sets.

3. Preserving Structural Invariants. For instance, on a degree-5 polynomial problem with $p(n) = n/(n^2 - 1)$ for $n \in \{2, \dots, 7\}$, monolithic SymCode produced an executable but disconnected polynomial and returned 39690 instead of $3/56$. The error was not numerical; SymCode failed to preserve the six interpolation constraints. SymPlan’s representation explicitly registered the six exact point conditions, guiding the plan to apply Lagrange interpolation with rational arithmetic before evaluating $p(8)$.

4. Boundaries of Structural Scaffolding. Semantic misformulation remains largely unchanged ($974 \rightarrow 969$, a negligible -0.5% shift). This provides a key scientific insight: neurosymbolic scaffolding effectively resolves interface alignment and execution failures, but cannot compensate when a compact model fundamentally lacks the underlying mathematical concept (as on non-constructive Level-5 problems). Integrating interactive formal theorem provers (e.g., Lean) represents the natural next frontier for extreme non-constructive mathematics.

V. Conclusion

We presented SymPlan, an inspectable neurosymbolic framework tailored for mathematical problem solving in compact small language models. By decoupling monolithic generation into a fused target-planning phase, contract-guided SymPy code synthesis, and targeted debugging with anti-regression ranking, SymPlan mitigates semantic misformulation, domain omission, and repair regressions. Across six diverse compact models (3B–8B) on the full MATH-500 benchmark under 4-bit quantization, SymPlan achieves uniform accuracy gains of $+3.60\%$ to $+5.20\%$ over SymCode and up to $+23.20\%$ over CoT, while improving Execution Success Rate to 89.90%, all without parameter updates. In future work, we plan to extend SymPlan to multimodal geometric diagrams and integrate formal proof verifiers on edge devices.

References

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.
- [2] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [3] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, and E. H. Chi, “Least-to-most prompting enables complex reasoning in large language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [4] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, vol. 1, 2023, pp. 2609–2634.
- [5] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, “Faithful chain-of-thought reasoning,” in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2023, pp. 305–329.
- [6] C. He, R. Luo, Y. Bai, S. Hu, Z. L. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang, J. Liu, L. Qi, Z. Liu, and M. Sun, “OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 4414–4443.
- [7] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “PAL: Program-aided language models,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, vol. 202, 2023, pp. 10 764–10 799.
- [8] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *Transactions on Machine Learning Research (TMLR)*, 2023.
- [9] K. Wang, H. Ren, A. Zhou, Z. Lu, S. Luo, W. Shi, R. Zhang, L. Song, M. Zhan, and H. Li, “MathCoder: Seamless code integration in large language models for enhanced mathematical reasoning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, “ToRA: A tool-integrated reasoning agent for mathematical problem solving,” in *International Conference on Learning Representations (ICLR)*, 2024.

- [11] S. B. Nezhad, Y. Li, and A. Agrawal, “SymCode: A neurosymbolic approach to mathematical reasoning via verifiable code generation,” arXiv preprint arXiv:2510.25975, 2025.
- [12] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh et al., “SymPy: Symbolic computing in Python,” *PeerJ Computer Science*, vol. 3, p. e103, 2017.
- [13] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, “Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning,” in *Findings of the Association for Computational Linguistics: EMNLP, 2023*, pp. 3806–3824.
- [14] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [15] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset,” in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [16] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 10 088–10 115.
- [17] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [18] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 46 534–46 594.
- [19] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, “Decomposed prompting: A modular approach for solving complex tasks,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [20] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” arXiv preprint arXiv:2402.03300, 2024.
- [21] A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, K. Lu, M. Xue, R. Lin, T. Liu, X. Ren, and Z. Zhang, “Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement,” arXiv preprint arXiv:2409.12122, 2024.
- [22] M. Abdin, S. A. Jacobs, A. A. Awan, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, H. Behl et al., “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv preprint arXiv:2404.14219, 2024.
- [23] Qwen Team, “Qwen2.5 technical report,” arXiv preprint arXiv:2412.15115, 2024.
- [24] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Dang et al., “Qwen2.5-Coder technical report,” arXiv preprint arXiv:2409.12186, 2024.
- [25] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan et al., “The Llama 3 herd of models,” arXiv preprint arXiv:2407.21783, 2024.